

IAM TASK

1. Create one IAM user and assign EC2 and S3 full access roles.

Step 1: Login to AWS Console

Explanation:

Login to AWS using root or admin account to create IAM users.

Step 2: Open IAM Service

Explanation:

IAM is used to manage users, groups, and permissions.

Step 3: Create new user named dev-user

Explanation:

IAM user represents a person or application that needs access.

Step 4: Enable console access

Explanation:

This allows user to login to AWS dashboard.

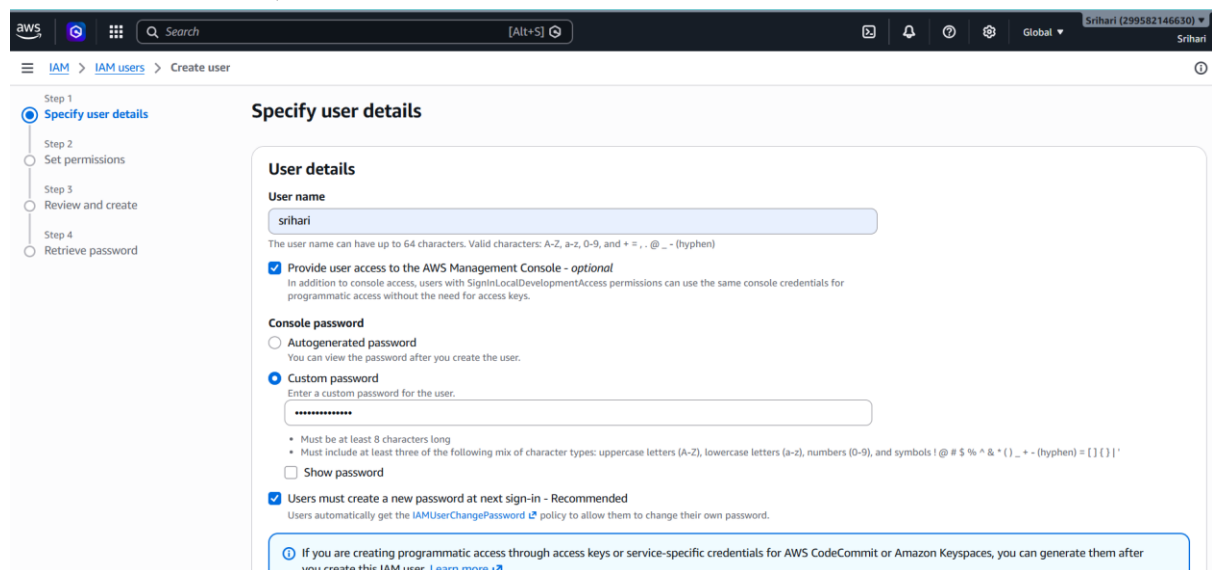
Step 5: Attach policies AmazonEC2FullAccess and AmazonS3FullAccess

Explanation:

Policies define permissions.

AmazonEC2FullAccess → full control over EC2

AmazonS3FullAccess → full control over S3



The screenshot shows the AWS IAM console interface for creating a new user. The breadcrumb navigation at the top reads 'IAM > IAM users > Create user'. On the left, a vertical list of steps is shown: Step 1 (Specify user details, active), Step 2 (Set permissions), Step 3 (Review and create), and Step 4 (Retrieve password). The main content area is titled 'Specify user details' and contains a 'User details' section. Under 'User name', the text 'srihari' is entered in a light blue input field. Below this, a note states: 'The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and +, =, ., @, _ - (hyphen)'. There are two checkboxes: 'Provide user access to the AWS Management Console - optional' (checked) and 'Users must create a new password at next sign-in - Recommended' (checked). Under the first checkbox, a note says: 'In addition to console access, users with SigninLocalDevelopmentAccess permissions can use the same console credentials for programmatic access without the need for access keys.' Under the 'Console password' section, 'Autogenerated password' is unselected, and 'Custom password' is selected. A password input field contains eight asterisks. Below the field, two requirements are listed: 'Must be at least 8 characters long' and 'Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ - (hyphen) = [] { } | ' '. A 'Show password' checkbox is unselected. At the bottom, a blue information box contains a note: 'If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. Learn more' with a link icon.

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

☐ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☒ Attach policies directly
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (2/1527)

Create policy

Choose one or more policies to attach to your new user.

Q s3fu

Filter by Type

All types

1 match

< 1 >

☒	Policy name	Type	Attached entities
☒	 AmazonS3FullAccess	AWS managed	0

► Set permissions boundary - optional

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Review and create

Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name
srihari

Console password type
Custom password

Require password reset
Yes

Permissions summary

Name	Type	Used as
AmazonEC2FullAccess	AWS managed	Permissions policy
AmazonS3FullAccess	AWS managed	Permissions policy
IAMUserChangePassword	AWS managed	Permissions policy

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

 User created successfully

You can view and download the user's password and email instructions for signing in to the AWS Management Console.

View user

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Retrieve password

You can view and download the user's password below or email users instructions for signing in to the AWS Management Console. This is the only time you can view and download this password.

Console sign-in details

Email sign-in instructions

Console sign-in URL
 <https://299582146630.signin.aws.amazon.com/console>

User name
 srihari

Console password
 ***** [Show](#)

Cancel

Download .csv file

Return to users list

2. Create one group in IAM and assign read access for EC2.

Step 1: Create group EC2-Read-Group

Explanation:

Group is used to manage multiple users with same permissions.

Step 2: Attach policy AmazonEC2ReadOnlyAccess

The screenshot shows the AWS IAM console interface. It starts with the 'Create new group' page where the 'User group name' is set to 'access_for_ec2'. Below this, the 'Add users to the group' section shows one user, 'srihari', has been added. The 'Attach permissions policies' section shows the 'AmazonEC2ReadOnlyAccess' policy attached to the group. A green notification bar at the top of the next page confirms 'access_for_ec2 user group created.' The final view shows the 'IAM user groups' list with the 'access_for_ec2' group listed with '1' user and 'Defined' permissions.

User group name
Enter a meaningful name to identify this group.

Maximum 128 characters. Use alphanumeric and '+=, @, _' characters.

Add users to the group - Optional (1/1) [Info](#)

An IAM user is an entity that you create in AWS to represent the person or application that uses it to interact with AWS.

< 1 >

☒	User name	Group	Last activity	Creation time
☒	srihari	0	-	4 minutes ago

Attach permissions policies - Optional (1/1179) [Info](#)

You can attach up to 10 policies to this user group. All the users in this group will have permissions that are defined in the selected policies.

Filter by Type All types 1 match < 1 >

☒	Policy name	Type	Used as	Description
☒	AmazonEC2ReadOnlyA...	AWS managed	None	Provides read only access to Amazon E...

access_for_ec2 user group created. View group

IAM user groups (1/1) [Info](#) Delete Create group

A user group is a collection of IAM users. Use groups to specify permissions for a collection of users.

< 1 >

☒	Group name	Users	Permissions	Creation time
☒	access_for_ec2	1	Defined	Now

Explanation:

This allows viewing EC2 but not modifying.

3. Create a new user named "Devops" and add to the group created in task 2.

Step 1: Create user Devops

Explanation:

Represents DevOps engineer.

Step 2: Add user to EC2-Read-Group

The screenshot displays the AWS IAM console interface for creating a new user. The top navigation bar shows the AWS logo, a search bar with 'iam', and a user profile for 'Srihari (299582146630)'. The main content area is divided into two sections: 'Specify user details' and 'Set permissions'.

Specify user details

User details

User name
Devops
The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , _ - (hyphen)

☒ **Provide user access to the AWS Management Console - optional**
In addition to console access, users with SigninLocalDevelopmentAccess permissions can use the same console credentials for programmatic access without the need for access keys.

Console password

☐ **Autogenerated password**
You can view the password after you create the user.

☒ **Custom password**
Enter a custom password for the user.

• Must be at least 8 characters long
• Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ + - (hyphen) = [] { } ' "
☐ Show password

☒ **Users must create a new password at next sign-in - Recommended**
Users automatically get the [IAMUserChangePassword](#) policy to allow them to change their own password.

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☒ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.

☐ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

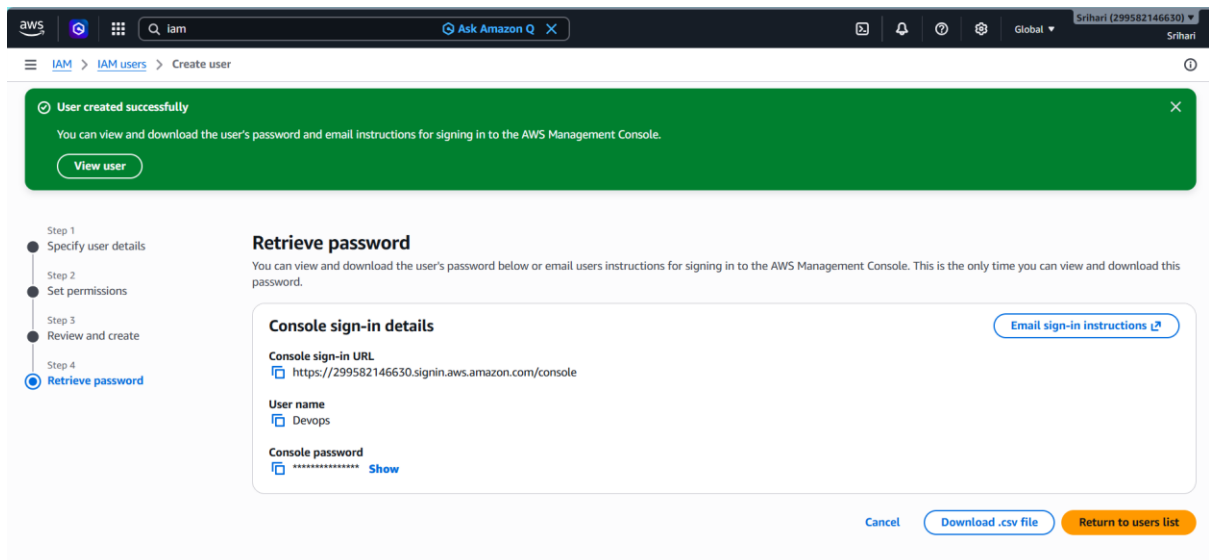
User groups (1/1)

Search

☒	Group name ↗	Users	Attached policies ↗	Created
☒	access_for_ec2	1	AmazonEC2ReadOnlyAccess	2026-07-20 (3 minutes ago)

Set permissions boundary - optional

Cancel Previous Next



Explanation:

User inherits permissions from group automatically.

4. Write a bash script to create an IAM user with VPC full access.

Step 1: Connect to the EC2 Instance

Connect to your Amazon Linux EC2 instance using SSH.

Step 2: Configure AWS CLI

Configure AWS CLI with your IAM user's credentials.

Step 3: Create the Bash Script

Create a new script file.

Step 4: Provide Execute Permission

```
chmod 755 IAMrole.bash
```

Step 5: Execute the Script

```

ec2-user@ip-172-31-34-209:~
#!/bin/bash
# ===== VARIABLES =====
IAM_USER="VpcUserDemo"
POLICY_ARN="arn:aws:iam::aws:policy/AmazonVPCFullAcce"
# ===== CREATE IAM USER =====
echo "Creating IAM user: $IAM_USER ..."
aws iam create-user --user-name $IAM_USER
# ===== CREATE ACCESS KEYS =====
echo "Creating access keys for $IAM_USER ..."
aws iam create-access-key --user-name $IAM_USER >
${IAM_USER}_creds.json
echo "Access keys saved in ${IAM_USER}_creds.json"
# ===== ATTACH POLICY =====
echo "Attaching VPC Full Access policy to $IAM_USER ..."
aws iam attach-user-policy --user-name $IAM_USER policyarn
$POLICY_ARN
echo "User $IAM_USER created successfully with VPC Full
Access."
~
~
~
~

```

```

[ec2-user@ip-172-31-34-209 ~]$ vi IAMrole.bash
[ec2-user@ip-172-31-34-209 ~]$ aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.18.38-73.137.amzn2023.x86_64 source/x86_64.amzn.2023
[ec2-user@ip-172-31-34-209 ~]$ aws configure
AWS Access Key ID [None]: AKIA3L4KGGBKF4BNZRZL
AWS Secret Access Key [None]: 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
Default region name [None]: eu-north-1
Default output format [None]: json
[ec2-user@ip-172-31-34-209 ~]$ vi IAMrole.bash
[ec2-user@ip-172-31-34-209 ~]$ vi IAMrole.bash
[ec2-user@ip-172-31-34-209 ~]$ chmod 755 IAMrole.bash
[ec2-user@ip-172-31-34-209 ~]$ ./IAMrole.bash
Creating IAM user: VpcUserDemo ...
{
  "User": {
    "Path": "/",
    "UserName": "VpcUserDemo",
    "UserId": "AIDA3L4KGGBKCA3ZBGZV3",
    "Arn": "arn:aws:iam::781436989524:user/VpcUserDemo",
    "CreateDate": "2026-07-21T10:15:27+00:00"
  }
}

```

5. Create an IAM policy to allow EC2 access for a specific user in specific regions only.

Step 1: Create new policy using JSON

Explanation:

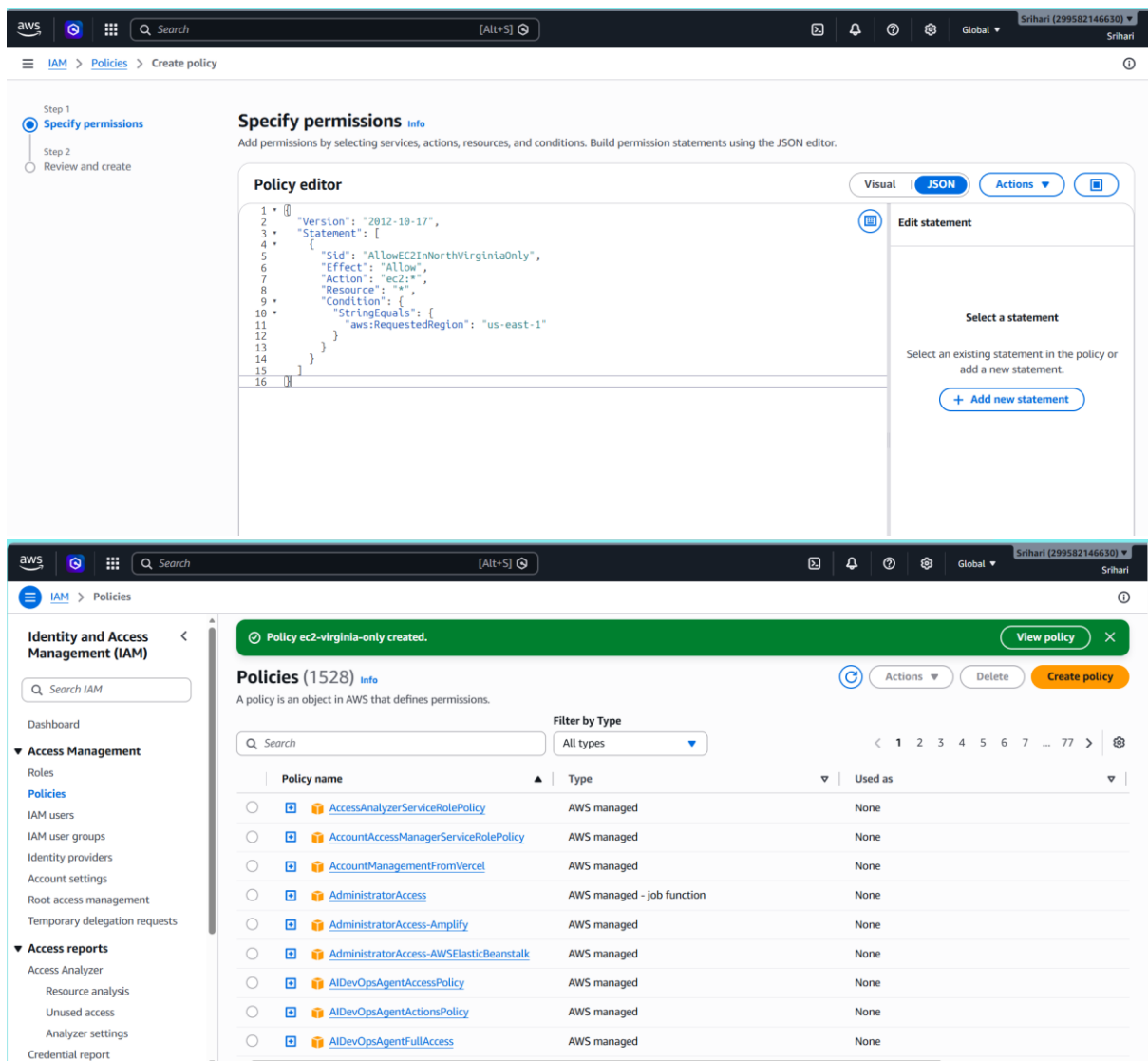
JSON defines permissions.

Step 2: Restrict region using condition

Explanation:

Allows access only in specific AWS region.

Step 3: Attach policy to user



The screenshot displays the AWS IAM console interface for creating a new policy. The top navigation bar shows the user is logged in as 'Srihari (299582146630)'.

Specify permissions

Step 1: Specify permissions (selected)
Step 2: Review and create

Policy editor

Visual JSON Actions

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Sid": "AllowEC2InNorthVirginiaOnly",  
6       "Effect": "Allow",  
7       "Action": "ec2:*",  
8       "Resource": "*",  
9       "Condition": {  
10        "StringEquals": {  
11          "aws:RequestedRegion": "us-east-1"  
12        }  
13      }  
14    }  
15  ]  
16 }
```

Policy editor

Visual JSON Actions

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

+ Add new statement

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Policy ec2-virginia-only created.

Policies (1528)

A policy is an object in AWS that defines permissions.

Filter by Type: All types

Policy name	Type	Used as
AccessAnalyzerServiceRolePolicy	AWS managed	None
AccountAccessManagerServiceRolePolicy	AWS managed	None
AccountManagementFromVercel	AWS managed	None
AdministratorAccess	AWS managed - job function	None
AdministratorAccess-Amplify	AWS managed	None
AdministratorAccess-AWSElasticBeanstalk	AWS managed	None
AIDevOpsAgentAccessPolicy	AWS managed	None
AIDevOpsAgentActionsPolicy	AWS managed	None
AIDevOpsAgentFullAccess	AWS managed	None

aws [Search] [Alt+S] Global Srihari (299582146630) Srihari

IAM > IAM users > srihari > Add permissions

Step 1 Add permissions
Step 2 Review

Add permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

- ☐ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.
- ☐ Copy permissions
Copy all group memberships, attached managed policies, inline policies, and any existing permissions boundaries from an existing user.
- ☒ Attach policies directly
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1525)

Filter by Type: All types 1 match

Policy name	Type	Attached entities
☒ ec2-virginia-only	Customer managed	0

Cancel Next

aws [Search] [Alt+S] Global Srihari (299582146630) Srihari

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

- Roles
- Policies
- IAM users**
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

Access reports

- Access Analyzer
- Resource analysis
- Unused access
- Analyzer settings
- Credential report

1 policy added

ARN: [arn:aws:iam::299582146630:user/srihari](#)

Console access: Enabled without MFA

Last console sign-in: Never

Access key 1: [Create access key](#)

Created: July 20, 2026, 18:08 (UTC+05:30)

Permissions Groups (1) Tags Security credentials Last Accessed

Permissions policies (5)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type: All types

Policy name	Type	Attached via
☐ AmazonEC2FullAccess	AWS managed	Directly
☐ AmazonEC2ReadOnlyAccess	AWS managed	Group access_for_ec2
☐ AmazonS3FullAccess	AWS managed	Directly
☐ ec2-virginia-only	Customer managed	Directly

Explanation:

User now can access EC2 only in allowed region

6. We have two accounts: Account A and Account B. Account A user should access an S3 bucket in Account B.

1. Overview

Cross-account role assumption allows one AWS account (Source Account) to access resources in another AWS account (Target Account) securely using IAM roles and temporary credentials.

Example:

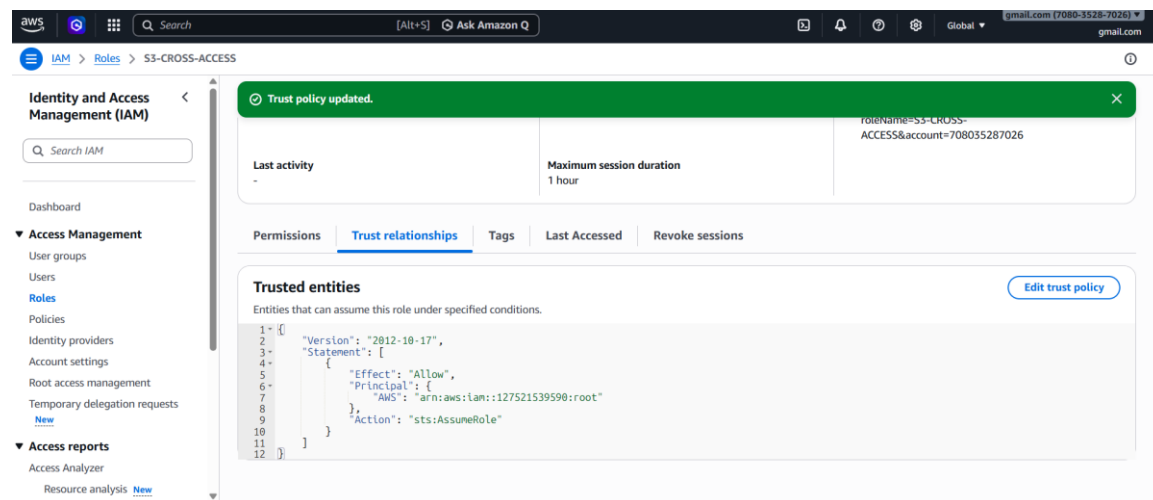
Source Account ID: 127521539590

Target Account ID: 708035287026

Role Name: friend-s3-access

Purpose:

This is used when you want to give secure access between different AWS accounts without sharing permanent credentials.



2. Key Components

Source Account:

The account that wants access.

Target Account:

The account that owns the resources.

IAM Role:

Created in Target Account. This role contains permissions (example: S3 access).

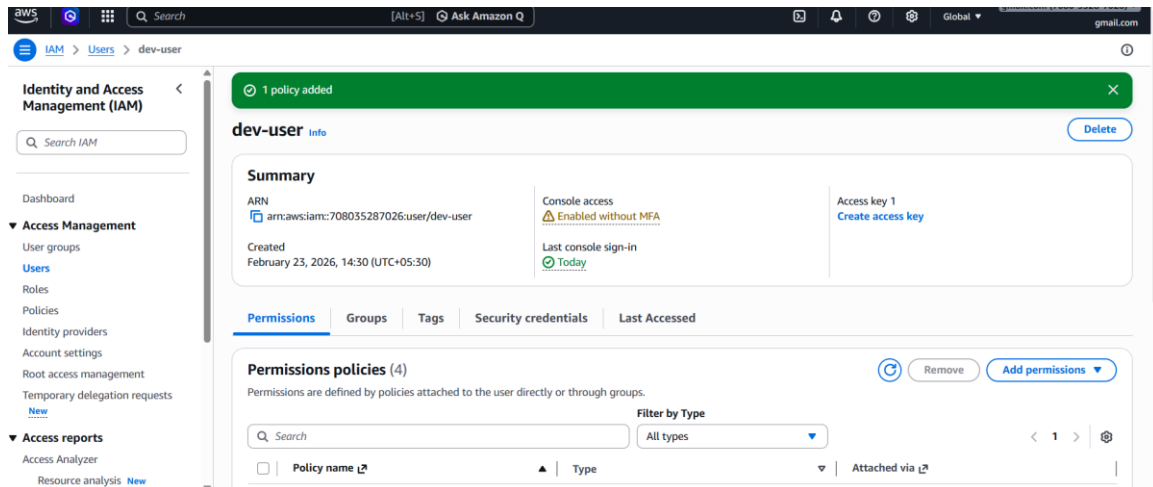
Trust Policy:

Defines WHO can assume the role.

Permission Policy:

Defines WHAT actions are allowed.

STS (Security Token Service):
Generates temporary credentials.



3. Step 1: Create Role in Target Account

Go to:
AWS Console → IAM → Roles → Create Role

Select:
Trusted Entity Type → AWS Account

Enter Source Account ID:
127521539590

Attach permissions:
Example: AmazonS3FullAccess

Role Name:
friend-s3-access

Click Create Role

4. Step 2: Configure Trust Policy

Go to:
IAM → Roles → friend-s3-access → Trust Relationships → Edit

Add:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::127521539590:root"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

This allows Source Account to assume the role.

5. Step 3: Add Permission in Source Account

Go to:

IAM → Users → Select user → Add Inline Policy

Add:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::708035287026:role/friend-s3-access"
    }
  ]
}
```

This allows user to assume the target role.

6. Step 4: Assume Role using AWS CLI

Command:

```
aws sts assume-role \
--role-arn arn:aws:iam::708035287026:role/friend-s3-access \
--role-session-name test
```

This generates temporary credentials.

7. Step 5: Export Temporary Credentials

```
export AWS_ACCESS_KEY_ID=XXXX
export AWS_SECRET_ACCESS_KEY=XXXX
export AWS_SESSION_TOKEN=XXXX
```

8. Step 6: Verify Access

aws sts get-caller-identity

Output should show Target Account ID.

```
ec2-user@ip-10-0-1-200-:~$ aws sts get-caller-identity
{
  "UserId": "AIDAZ2W8RC7ZTJGU3RTY",
  "Account": "70803287026",
  "Arn": "arn:aws:iam::70803287026:user/pixelcloud-s3-uploader"
}

ec2-user@ip-10-0-1-200-~$ aws sts assume-role \
--role-arn arn:aws:iam::132483690424:role/friend-s3-access \
--role-session-name test

{
  "Credentials": {
    "AccessKeyId": "ASIAR3WFV64FJNNUXW",
    "SecretAccessKey": "c3h8c211qwhm9v5PT12Lxk7ruakDOAtxIsnf",
    "SessionToken": "T0q3b13p27lxb2v3JC6cXV2LWhc31tKS1MEVC1QDR9Y2P1whddgVQPt9u0p6q0w10wz27puy3Jef01fHAK5SR/VBvYb960zc0xhvc0j8v2kMtufuJp9AgNmkdtkpoCpDP////////wEQABwMTHYAdgzNjkwNDI0gm0bYcpalZOZduV9yqG2h0yYmCCEt001q4j0SCeZr8jYy9u6FteEvXtCJCfE01tPHp3JopRCcCp04774tGKE3q01c2wagvEqDmH31ALRg0S11HvYy9H8rAY5jv8JkTm0x0vrlpahbV4N8wGmgncRd7qL3Ckqgd905-f1dv1YkS10JRUZVC0ahmUxvJc/N/5/Z1R1vYHPBq5cJPZ7wAwKtK3Zul6cwQ1S9m0SeCmHfF3Gt9m0qdcS1t9uq3ALV3k01F9wggm4hQj07yeh0u1AYtCz4b3C7F7zpg9yGm0p6c0p6tW8R2kukh1h3js/nq2t0yYv-bV7ZednAcQZ3HP/wskRr/dISN0wApRtKCSN9P13K1U0Y0JC1U0P1apZV4cnd+PuPHp++2N3x3cPAl0NgmAKwC1/g4rH3pZ2P06/cdnw1onDAL0fK8Wk8N4BhBn3OPbMR5c+kQ+yryk+Q5RCBUZvZ2hm2Q64T1LEnE60P4",
    "Expiration": "2026-02-24T08:16:59:00:00"
  },
  "AssumedRoleUser": {
    "AssumedRoleId": "AR0ASWtE64LQNS46NP: test",
    "Arn": "arn:aws:sts::132483690424:assumed-role/friend-s3-access/test"
  }
}
```

[illegible]

Result:

User in Account A can access S3 bucket in Account B.

1) Common Errors

Access Denied:

Missing trust policy or permission policy

Invalid Credentials:

Wrong export of temporary credentials

2). Security Advantage

No need to share permanent credentials

Uses temporary credentials

More secure access control

Conclusion

- IAM user creation
- Group management
- Policy management
- Automation using bash
- Custom policies
- Cross account access